MATH6005: Python Assignment 2: Shortest Path using Dijkstra's Algorithm

Ahmet Engin ADIYAMAN

36536989

25/11/2024

Table of Contents

MATH6005: Python Assignment 2: Shortest Path using Dijkstra's Algorithm 1
Detailed Explanation of the Algorithm 3
Elements 3
Loops 3
Efficiency of the Algorithm 4
Good Coding Practice 4
Ease of Use & Applying Algorithm To Another Network 4
Efficiency Adjustment 4
Limitations 5
Applied Tests 5
Test 1 5
Test 2 5
Test 3 5
Test 4 6
Appendix A 7

Calculating Shortest Path Using Dijkstra's Algorithm on Python

The first step of coding an algorithm that can find the shortest path for a given network diagram using Dijkstra Algorithm was to start planning. After reading provided instructions again and again, I still had some questions like, How does the cutting algorithm handle ties? How to make algorithm choose the shortest path in tie situations?

To simulate the situation and understand the task better, I have made some background research about how Dijkstra algorithm works. Next step was to work on a simple simulation on paper before start coding [See Appendix A]. This helped understanding how I can establish an algorithm that can work accurately and efficiently, as well has how to transform ideas in my head to tangible algorithms.

Detailed Explanation of the Algorithm

After importing relevant library elements, the algorithm starts by importing data from an excel worksheet. The excel worksheet contains 3 columns. These are From, To, Duration. Basically, this is a list that transform 3D network diagram into 2D tangible data set. What makes this an efficient way to import data is that Microsoft Excel is one of the most popular data sharing software, which enables the algorithm to work on different data sets with ease.

Next, required elements are defined before the loop.

Elements

- **start** and **destination**: Two string elements being used as input for the algorithm to work.
- A dataframe named **df**, containing 3 columns:
    - **City**: all the unique cities given in the dataset.
    - **Y**: Shortest time for each city & station.
    - **P**: Shortest path for each city & station.
- **S**: An ampty list for visited cities
- **Y**: Total time spent, which is recorded for **df["Y"]** in each iteration.
- **unvisited**: List of unvisited cities to be able to loop
- **current_city**: The station or city where the loop is actively examining.
- A dataframe named **possibleCities**, containing 2 columns:
    - **neighbor**: All the possible ways out of the examined station.
    - **duration**: All the possible relevant durations out of the examined station.
- **result**: Which is equal to df's destination row to grab relevant Y and P values.

Loops

The algorithm works based on 2 different loops. A **while loop** and a **for loop** within **while loop**. While the first **while loop** iterates over each unvisited city until the algorithm visits all cities, **for loop** iterates over the rows of **possibleCities** to choose the next step, find the station with minimum **Y** value, and record & update **df["Y"]** and **df["P"]** elements.

The algorithm starts within a **while loop** from **start** station defined at the beginning, and appends it to **S**, and removes it from **unvisited** list. Next, checks all possible stations going out of current station in **possibleCities**. After initialising the first step, a for loop iterates over

rows of **possibleCities** to find the minimum durations and shortest paths, at the same time, updates each **df** element for individual Y(j) and P(j) values.

The algorithm stops when all stations are visited, and returns **df["Y"]** minimum time to reach to requested **destination**, as well as the **df["P"]** value for the shortest path to reach to the requested **destination**.

Efficiency of the Algorithm

Good Coding Practice

The code is structured with a logical order;

1. library imports,
2. data imports,
3. initialisation
4. While loop and for loop
5. Exporting & Outputs

- ✓ Each section is explained with a comment beforehand, just like a header to give reader an overall idea about what to expect.
- ✓ Instead of manually calculating durations for each step, **while loop** and **for loop** are preferred to optimize processes.

Ease of Use & Applying Algorithm To Another Network

- ✓ It is easy to use because code is well structured in a logical order, and it contains many comments explaining each steps. Apart from that, it is easy to integrate this code to new algorithms as it imports data from Excel which is by far the most popular algorithm with rich compatibility & integration with other statistics, database, data visualization software. This enables it to be easily replaced with a new network.
- ✓ The whole algorithm is written within a function named shortest_path as;

```
def shortest_path(start, destination, file_name="data.xlsx"):
```

For that reason, it can be used quickly with a new set of 2 simple inputs.

Efficiency Adjustment

For the efficiency of the algorithm, a stopping condition is at the end of while loop.

```
#For the efficiency of the code, A stopping condition is set for
efficiency when destination is reached with the shortest possible
way.

if current_city == destination:

    break
```

Basically, when destination is reached with the shortest possible way. The loop breaks, and outputs the result.

In theory, this is against the stopping condition Dijkstra Algorithm defined, but it stops algorithm to calculate each and every route repetitively for even neighbour stations.

Considering this algorithm can be used with larger networks, this peace of code would definitely shorten calculation time.

Limitations

Dijkstra Algorithm does not seem to have a method to handle ties. After detailly reading the instructions, examining the given 'worked example', I found out that handling ties has no strict method for the algorithm. Understanding how to handle ties when durations are equal was challenging because I was not able to simulate it on paper or in my mind. After defining the **for loop** to calculate minimum durations, and revise paths accordingly.

Applied Tests

Test 1

```
#Trial #1
start = "MOSCOW"
destination = "SAINT PETERSBURG"
shortest_path(start, destination)
```

```
Starting station: MOSCOW
Destination station: SAINT PETERSBURG
Time: 1.0
Route: MOSCOW → SAINT PETERSBURG
```

Figure 1: A screenshot containing the first trial inputs and results.

Moscow station has 5 different way outs, 4 of them having the same distance. Here, we are testing if algorithm can select the shortest path in a tie situation.

- ✓ The test result is positive that the initial step works effectively, and it handles tie situations correctly.

Test 2

```
#Trial #2
start = "TROMSO"
destination = "ISTANBUL"
shortest_path(start, destination)
```

```
Starting station: TROMSO
Destination station: ISTANBUL
Time: 7.0
Route: TROMSO → OULU → SAINT PETERSBURG → KYIV → ISTANBUL
```

Figure 2: A screenshot containing the second trial inputs and results.

Tromso and **Istanbul** are stations with the most far distances.

- ✓ Making this test approves how the algorithm handles long paths, handle tie situations effectively, manage routes with tunnels, and successfully outputs the whole result for a trial with large path.

Test 3

```python
#Trial #3
start = "MADRID"
destination = "MURMANSK"
shortest_path(start, destination)
```

```
Starting station: MADRID
Destination station: MURMANSK
Time: 11.0
Route: MADRID → LONDON → HAMURG → STOCKHOLM → OULU → MURMANSK
```

Figure 3: A screenshot containing the first third trial inputs and results.

Between **Madrid** and **Murmanks** stations, we have many stations on the route that are shown bold, meaning there is a tunnel and duration to pass 1 rectangle is doubled.

- ✓ The test proves that the algorithm is working effectively against bold rectangles, meaning it correctly avoids tunnels when necessary.

Test 4

```python
#Trial #4
start = "EDINBURG"
destination = "EDINBURG"
shortest_path(start, destination)
```

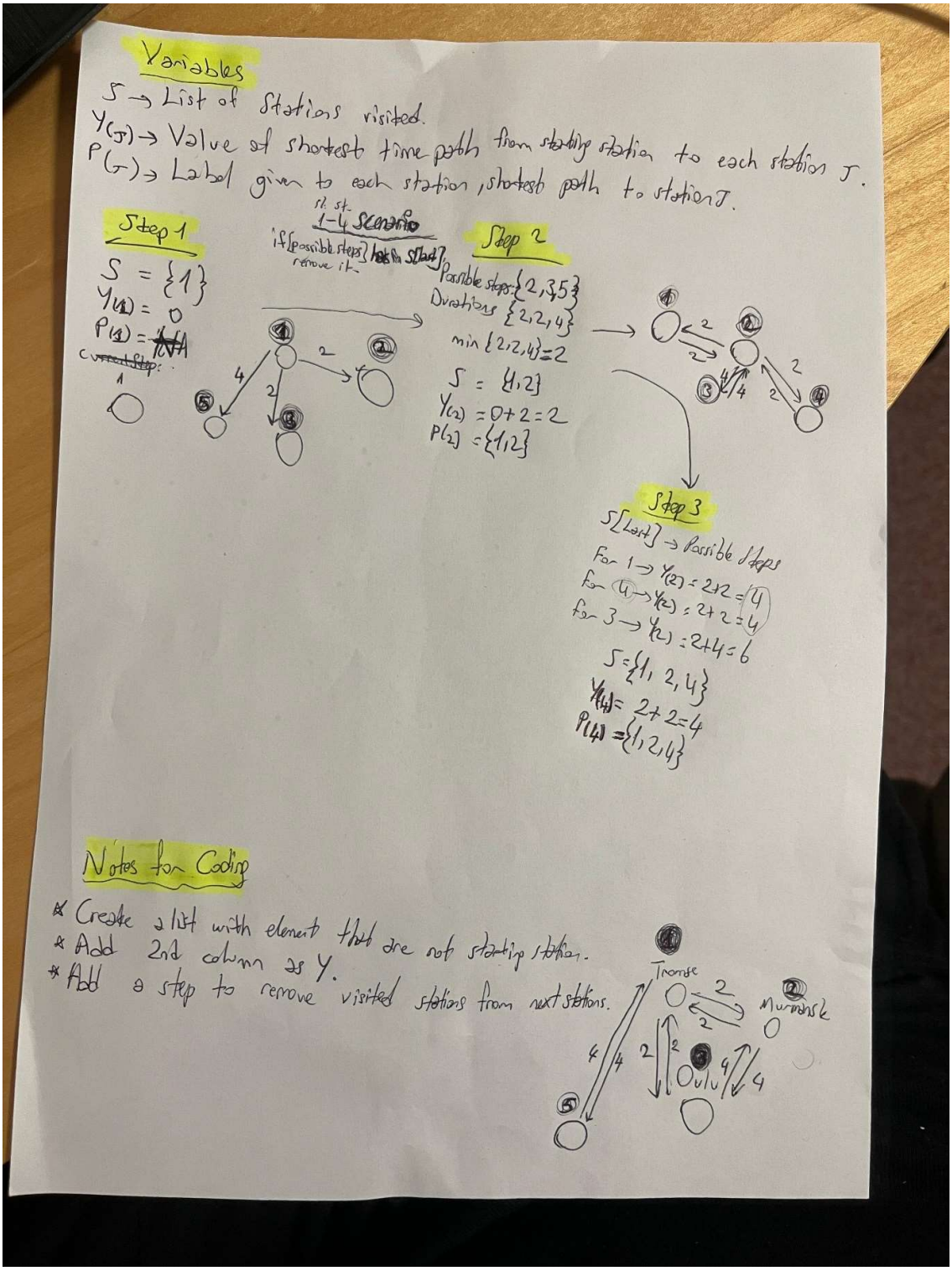
```
Error: Destination cannot be the same as starting point!
An exception has occurred, use %tb to see the full traceback.

SystemExit
C:\Users\Engin-Eer\anaconda3\Lib\site-packages\IPython\core\interactiveshell.py:3585: UserW
arning: To exit: use 'exit', 'quit', or Ctrl-D.
  warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)
```

Figure 4: A screenshot containing the fourth trial inputs and results.

Here, we are testing a situation when the user sets a wrong input by accident. In such case, the algorithm returns the wanted text message, saying “Error: Destination cannot be the same as starting point!”.

Appendix A


Variables
S → List of Stations visited.
Y(J) → Value of shortest time path from starting station to each station J.
P(J) → Label given to each station, shortest path to station J.
Step 1
S = {1}
Y(1) = 0
P(1) = N/A
Step 2
Possible steps: {2,3,5}
Durations {2,2,4}
min {2,2,4} = 2
S = {1,2}
Y(2) = 0+2 = 2
P(2) = {1,2}
Step 3
S[Last] → Possible steps
For 1 → Y(2) = 2+2 = 4
For 4 → Y(2) = 2+2 = 4
For 3 → Y(2) = 2+4 = 6
S = {1, 2, 4}
Y(4) = 2+2 = 4
P(4) = {1,2,4}
Notes for Coding
* Create a list with element that are not starting station.
* Add 2nd column as Y.
* Add a step to remove visited stations from next stations.
Tromsø
Murmansk
Oulu